



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Arquitecturas Distribuidas desde el punto de vista del Software

Unidad 3 · Aplicaciones Distribuidas (ISR-701) · Séptimo Semestre · Ingeniería de Software

Resultado de aprendizaje y ruta de la sesión



Resultado de aprendizaje

Aplicar arquitecturas y patrones de diseño distribuido —microservicios, interfaces REST, mensajería asíncrona y arquitectura en capas— para construir sistemas escalables, disponibles y seguros.

Sesión preparatoria del taller FORM-TL-03 (contrato OpenAPI) y de la práctica GA-SUM-02 (microservicios con Docker).

1. Qué es la arquitectura de software de un sistema distribuido
2. Estilos: cliente—servidor, capas, monolito, SOA y microservicios
3. Arquitectura orientada a eventos y mensajería asíncrona
4. Patrones: puerta de enlace, datos distribuidos y comunicación
5. Atributos de calidad y criterios de elección
6. Aplicación directa al Proyecto Fin de Curso (PFC)

¿Qué es la arquitectura de software de un sistema distribuido?

Es el conjunto de **decisiones estructurales** sobre el software: qué componentes existen, qué responsabilidad tiene cada uno y cómo se comunican (conectores), independientemente del hardware donde se ejecuten.



Vista física (del sistema)

Nodos, computadoras, redes y protocolos de transporte.

Responde: ¿dónde se ejecuta cada parte?

Ejemplo: tres servidores y un balanceador de carga.



Vista lógica (del software)

Componentes, módulos, servicios, contratos e interfaces.

Responde: ¿cómo se organiza y se comunica el código?

Ejemplo: servicio de autenticación, servicio de recursos y sus contratos.

Hoy trabajamos la vista lógica: el mismo software puede desplegarse en una o en cien máquinas sin cambiar su arquitectura lógica.

Cliente–Servidor: el punto de partida



Roles asimétricos: el cliente inicia la conversación; el servidor escucha, procesa y responde. Es la base de la web (protocolo HTTP, Protocolo de Transferencia de Hipertexto) y de toda interfaz de programación de aplicaciones (API).

Fortaleza: centraliza datos y lógica; simple de razonar.

Limitación: el servidor concentra la carga y es un punto único de fallo si no se replica.

Arquitectura en capas (niveles)



Regla de oro: cada capa solo conversa con su capa adyacente. El acoplamiento queda controlado y cada capa puede cambiarse sin afectar al resto.

Distribución física: cuando cada capa se despliega en una máquina distinta hablamos de niveles (tiers): dos niveles, tres niveles o n niveles.

Vigencia: sigue viva dentro de cada microservicio: controlador, servicio y repositorio son capas.

El monolito: una sola unidad de despliegue

Aplicación monolítica

Módulo ventas

Módulo inventario

Módulo usuarios

Módulo reportes

Un solo ejecutable · una sola base de datos · un solo despliegue



Ventajas

Simplicidad inicial: un proyecto, un despliegue.

Transacciones locales: consistencia inmediata.

Depuración y pruebas de extremo a extremo sencillas.

Latencia mínima: llamadas en memoria.



Desventajas

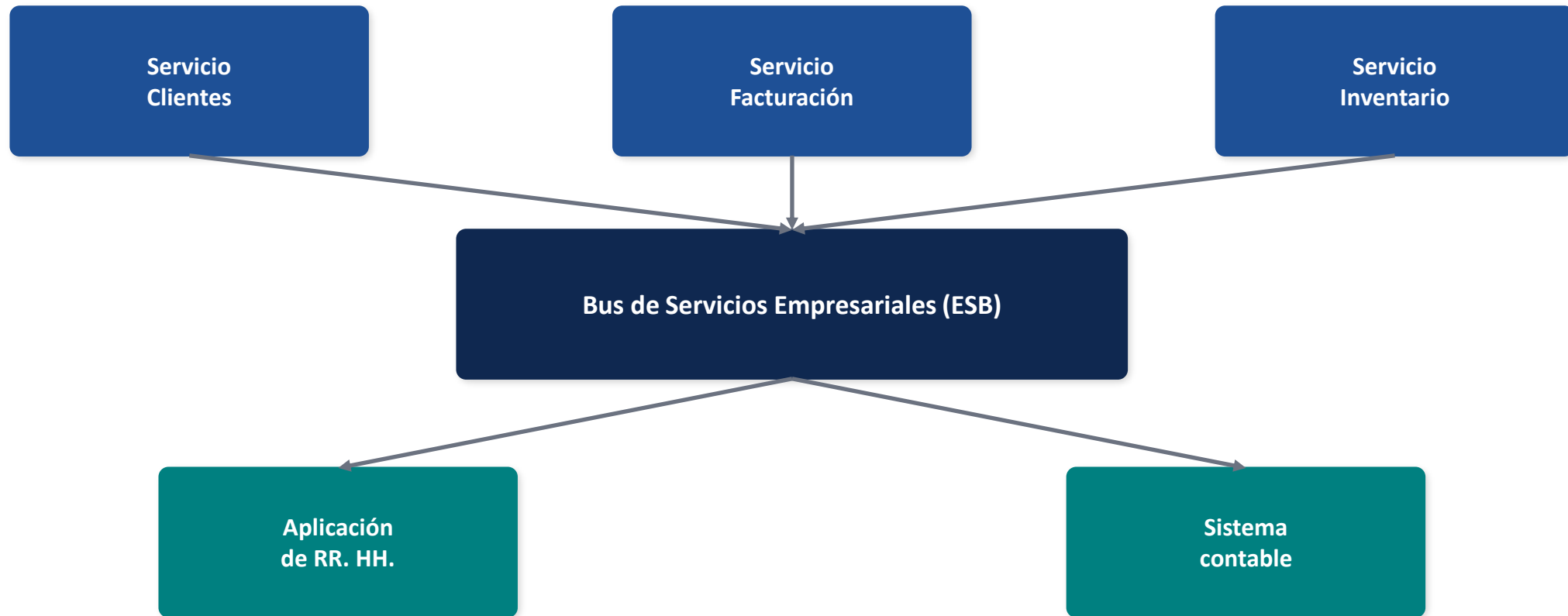
Escalado en bloque: se replica todo, aunque solo un módulo lo necesite.

Acoplamiento creciente con el tamaño del equipo.

Un despliegue arriesgado: cualquier cambio publica todo.

Atadura tecnológica a un solo lenguaje.

SOA: Arquitectura Orientada a Servicios



Idea central: exponer las capacidades de negocio como servicios reutilizables con contratos formales, integrados a través de un bus central que enruta, transforma y orquesta los mensajes. **Riesgo histórico:** el bus concentra lógica y se convierte en cuello de botella y punto único de fallo.

Microservicios: autonomía alrededor del negocio



Capacidad de negocio

Cada servicio modela una capacidad: pagos, catálogo, envíos.



Despliegue independiente

Se publica un servicio sin tocar ni detener a los demás.



Datos descentralizados

Cada servicio es dueño exclusivo de su base de datos.



Comunicación ligera

Contratos REST o mensajería; nada de bus inteligente central.



Equipos pequeños

Un equipo posee el servicio de punta a punta (construye y opera).



Tolerancia a fallos

El fallo de un servicio se aísla; el sistema se degrada, no muere.

«Servicios pequeños y autónomos que trabajan juntos, modelados alrededor de un dominio de negocio» — S. Newman

Comparativa: monolito, SOA y microservicios

Criterio	Monolito	SOA (Arq. Orientada a Servicios)	Microservicios
Unidad de despliegue	Una sola aplicación	Servicios grandes + bus central	Muchos servicios pequeños
Escalado	En bloque (todo o nada)	Por servicio, limitado por el bus	Fino: solo el servicio saturado
Datos	Base de datos única compartida	Frecuentemente compartidos	Una base por servicio
Tecnología	Un solo lenguaje/plataforma	Heterogénea vía el bus	Libre por servicio (políglota)
Fallo parcial	Derriba todo el sistema	El bus es punto único de fallo	Aislado por servicio
Complejidad operativa	Baja	Media-alta (gobierno del bus)	Alta: red, observabilidad, datos

No hay un ganador universal: hay contextos. La complejidad operativa de los microservicios solo se paga cuando sus beneficios se necesitan.

Arquitectura orientada a eventos (EDA)

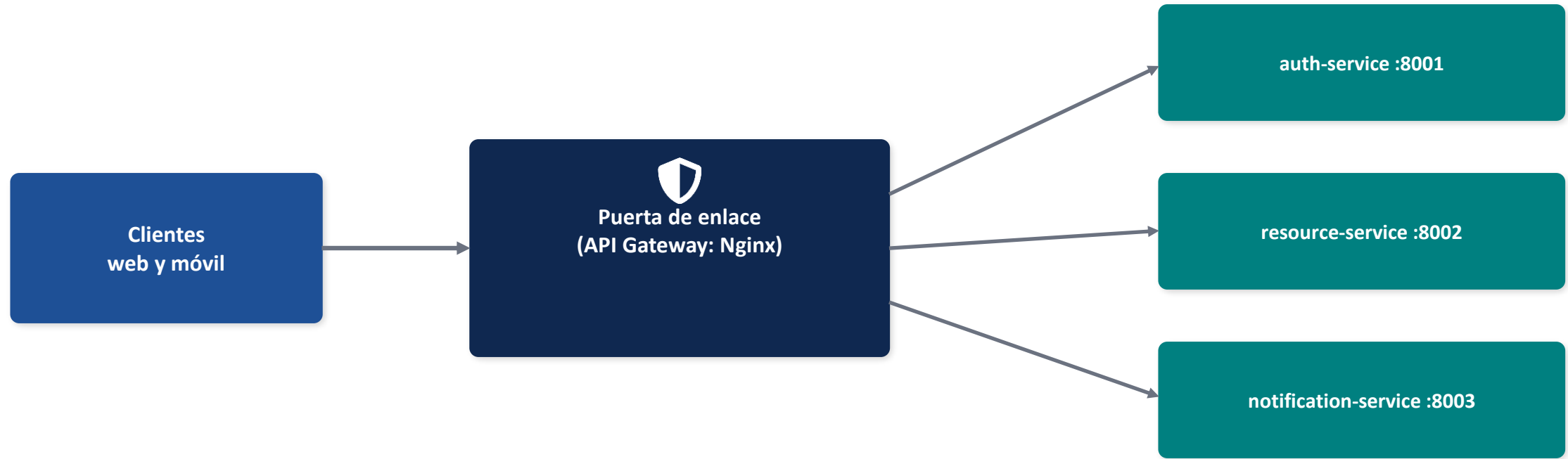


El productor publica un hecho del negocio y sigue trabajando **sin esperar respuesta** (desacoplamiento temporal). Los consumidores se suscriben y reaccionan a su propio ritmo.

Beneficios: absorbe picos de carga, agrega consumidores sin tocar al productor, tolera caídas temporales.

Costo: consistencia eventual: los datos convergen, pero no son idénticos en todo instante.

Patrón puerta de enlace (API Gateway)



Responsabilidades: punto único de entrada · enrutamiento por ruta · verificación del token de seguridad · límite de peticiones por minuto · registro centralizado de accesos

Cuidado: si concentra lógica de negocio, recrea el problema del bus de SOA; debe mantenerse delgado y replicable.

Patrones de datos distribuidos



Una base por servicio

Database per Service: cada microservicio es dueño exclusivo de sus datos; nadie más los consulta directamente.

Garantiza autonomía y despliegue independiente.

Consecuencia: ya no existen las uniones (JOIN) entre servicios.



CQRS

Segregación de Responsabilidad entre Comandos y Consultas: un modelo optimizado para escribir y otro distinto para leer.

Útil cuando las lecturas superan masivamente a las escrituras.

Costo: sincronizar ambos modelos (eventos).



Saga

Transacción distribuida como secuencia de transacciones locales.

Si un paso falla, se ejecutan compensaciones que deshacen los pasos anteriores.

Reemplaza a la transacción global, imposible entre bases independientes.

Los tres patrones responden a la misma decisión raíz: descentralizar los datos para ganar autonomía.

¿Cómo conversan los servicios? REST, gRPC y mensajería

Criterio	REST (Transferencia de Estado Representacional)	gRPC (llamada a procedimiento remoto)	Mensajería asíncrona
Estilo	Síncrono, recursos sobre HTTP	Síncrono, funciones sobre HTTP/2	Asíncrono, eventos vía broker
Formato	Texto JSON (legible)	Binario Protocol Buffers (compacto)	JSON o binario en colas
Contrato	OpenAPI 3.0	Archivo .proto	Esquema del evento
Acoplamiento	Temporal: ambos deben estar vivos	Temporal: ambos deben estar vivos	Mínimo: desacoplados en el tiempo
Uso típico	API pública hacia clientes	Comunicación interna de alta frecuencia	Notificaciones, integración, picos

Regla práctica: REST hacia afuera, gRPC o mensajería hacia adentro, y eventos para todo lo que pueda esperar.

Atributos de calidad que guían la arquitectura (ISO/IEC 25010)



Escalabilidad

Capacidad de crecer ante la demanda. Horizontal: más réplicas. Vertical: máquina más grande. Los microservicios habilitan el escalado fino.



Disponibilidad

Porcentaje de tiempo operativo. Se logra con redundancia: réplicas detrás de un balanceador; sin punto único de fallo.



Tolerancia a fallos

El sistema sigue (degradado) ante fallos parciales: tiempos de espera, reintentos y cortacircuitos (circuit breaker).



Desplegabilidad

Frecuencia y seguridad con la que se publica software: despliegues pequeños, independientes y reversibles.

¿Cómo elegir la arquitectura? Criterios honestos

1

Tamaño y madurez del equipo

Pocos desarrolladores y dominio incierto: monolito modular. Varios equipos autónomos: microservicios.

2

Frecuencia de despliegue

¿Se necesita publicar partes del sistema varias veces al día sin detener el resto? Solo entonces el despliegue independiente paga su costo.

3

Fronteras del dominio

Los servicios se cortan por capacidades de negocio bien delimitadas (Diseño Guiado por el Dominio), nunca por capas técnicas.

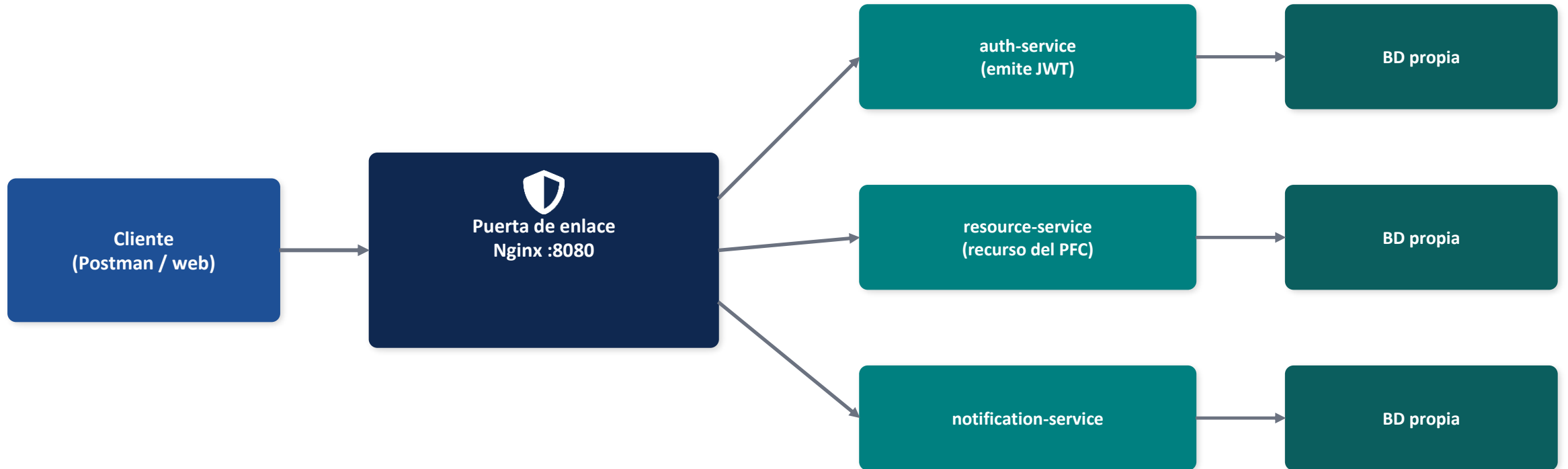
4

Capacidad operativa

Microservicios exigen contenedores, observabilidad y automatización; sin esa base, son un monolito distribuido: lo peor de ambos mundos.

Consejo de Newman: empezar con un monolito modular y migrar a microservicios cuando el dolor sea real y localizado.

Esta arquitectura es la de su Proyecto Fin de Curso



Microservicios + puerta de enlace + una base por servicio + token JWT + Docker Compose = **Entrega 2 del PFC (GA-SUM-02)**. El contrato REST de cada servicio se documenta en OpenAPI 3.0 en el taller FORM-TL-03.

Glosario de siglas de la sesión

Sigla	Significado	Sigla	Significado
API	Interfaz de Programación de Aplicaciones (Application Programming Interface)	ESB	Bus de Servicios Empresariales (Enterprise Service Bus)
REST	Transferencia de Estado Representacional (Representational State Transfer)	EDA	Arquitectura Orientada a Eventos (Event Driven Architecture)
HTTP	Protocolo de Transferencia de Hipertexto (HyperText Transfer Protocol)	CQRS	Segregación de Responsabilidad entre Comandos y Consultas
JSON	Notación de Objetos de JavaScript (JavaScript Object Notation)	JWT	Token Web de JSON (JSON Web Token)
SOA	Arquitectura Orientada a Servicios (Service Oriented Architecture)	gRPC	Llamada a procedimiento remoto de Google (Google Remote Procedure Call)
PFC	Proyecto Fin de Curso	ISO/IEC	Organización Internacional de Normalización / Comisión Electrotécnica Internacional

Referencias (norma IEEE)

- [1] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2.^a ed. Sebastopol, CA, EE. UU.: O'Reilly Media, 2021. ISBN: 978-1-4920-3402-5.
- [2] C. Richardson, Microservices Patterns: With Examples in Java. Shelter Island, NY, EE. UU.: Manning Publications, 2018. ISBN: 978-1-617-29454-1.
- [3] U. Joshi, Patterns of Distributed Systems. Boston, MA, EE. UU.: Addison-Wesley Professional, 2023. ISBN: 978-0-13-822198-0.
- [4] G. Blinowski, A. Ojdowska y A. Przybylek, «Monolithic vs. microservice architecture: A performance and scalability evaluation», IEEE Access, vol. 10, pp. 20357–20374, 2022. doi: 10.1109/ACCESS.2022.3152803.
- [5] I. Karabey Aksakalli, T. Çelik, A. B. Can y B. Tekinerdoğan, «Deployment and communication patterns in microservice architectures: A systematic literature review», Journal of Systems and Software, vol. 180, art. 111014, oct. 2021. doi: 10.1016/j.jss.2021.111014.
- [6] R. T. Fielding, «Architectural styles and the design of network-based software architectures», tesis doctoral, Univ. de California, Irvine, CA, EE. UU., 2000.



Síntesis y preguntas

- La arquitectura de software distribuida es un conjunto de decisiones sobre componentes y conectores.
- Los estilos evolucionaron del cliente–servidor al monolito, a SOA y a los microservicios: cada uno cambia dónde vive el acoplamiento.
- Los patrones (puerta de enlace, una base por servicio, CQRS, Saga) hacen operable la descentralización.
- Se elige con atributos de calidad y contexto, no con modas.
- **Su Proyecto Fin de Curso implementa, pieza por pieza, esta arquitectura.**

Próxima actividad: taller FORM-TL-03 — diseño y documentación del contrato REST con OpenAPI 3.0.